

EE 446 TinyML Lab 6

Quantized Anomaly Detection on Arduino Nano 33 BLE Sense

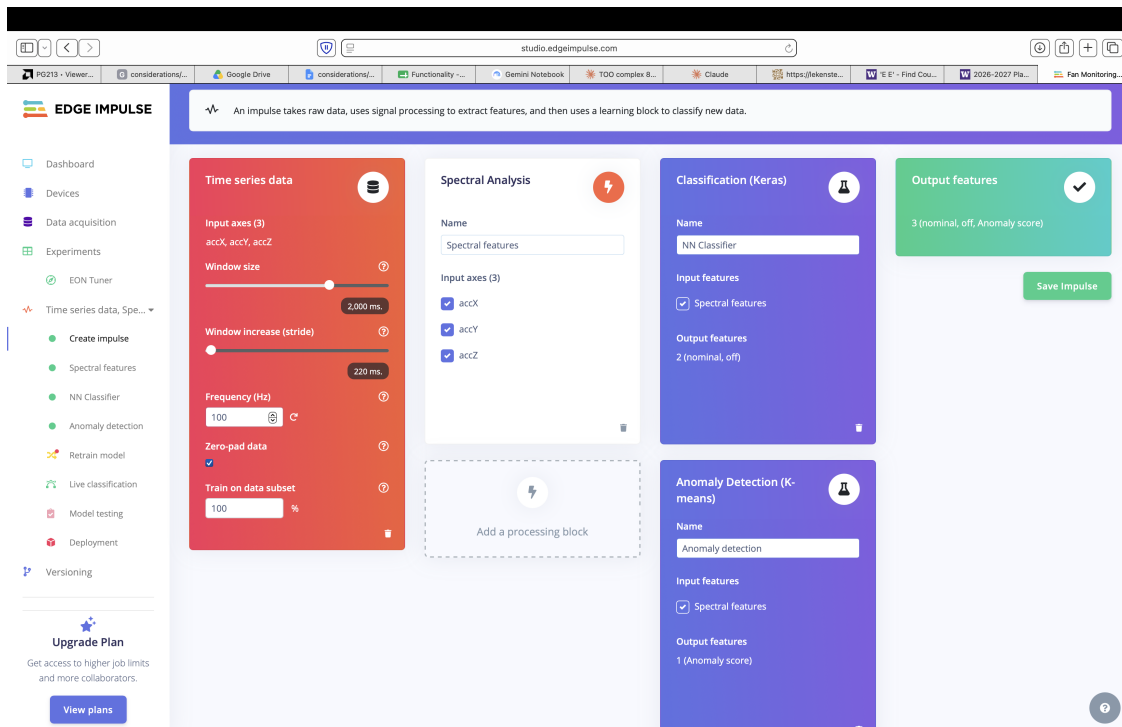
July 2026

1 Edge Impulse Results

Public Edge Impulse project link: <https://studio.edgeimpulse.com/public/1074077/live>

Raw inputs

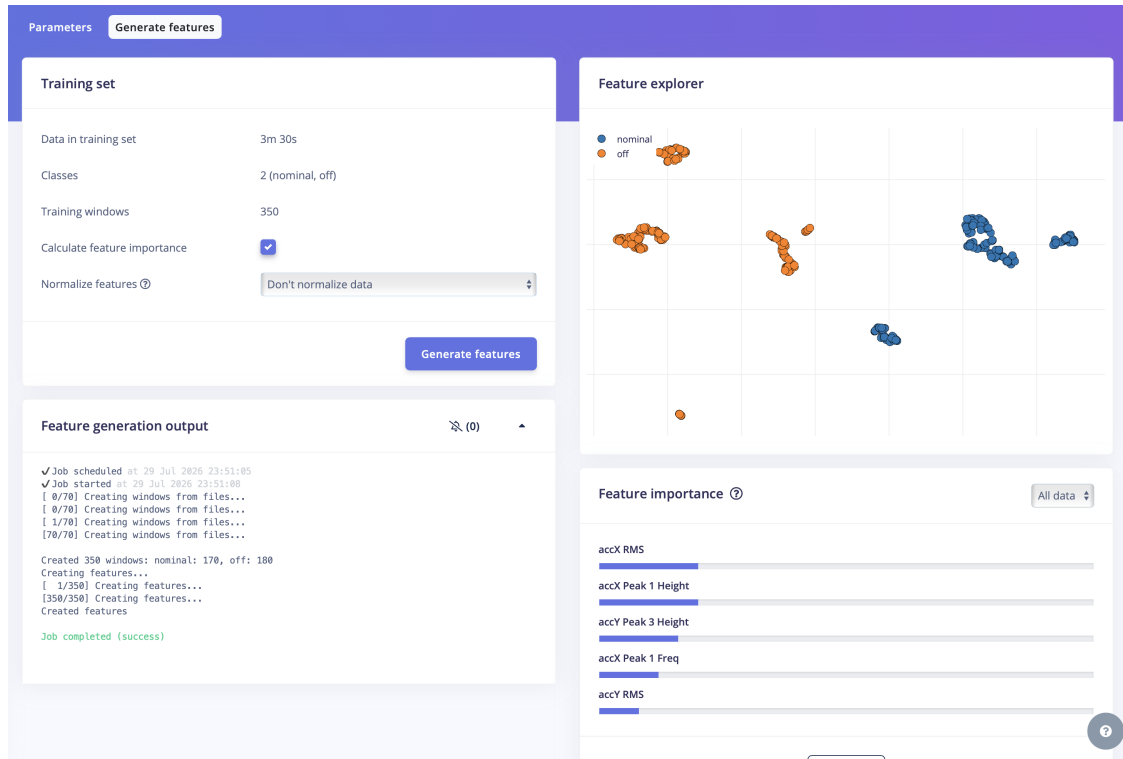
The project uses three-axis accelerometer data with the axes `accX`, `accY`, and `accZ`. The time series block samples at 100 Hz with a window size of 2000 ms and a window increase of 220 ms, zero-pads short windows, and trains on 100% of the data subset.



NN classifier features

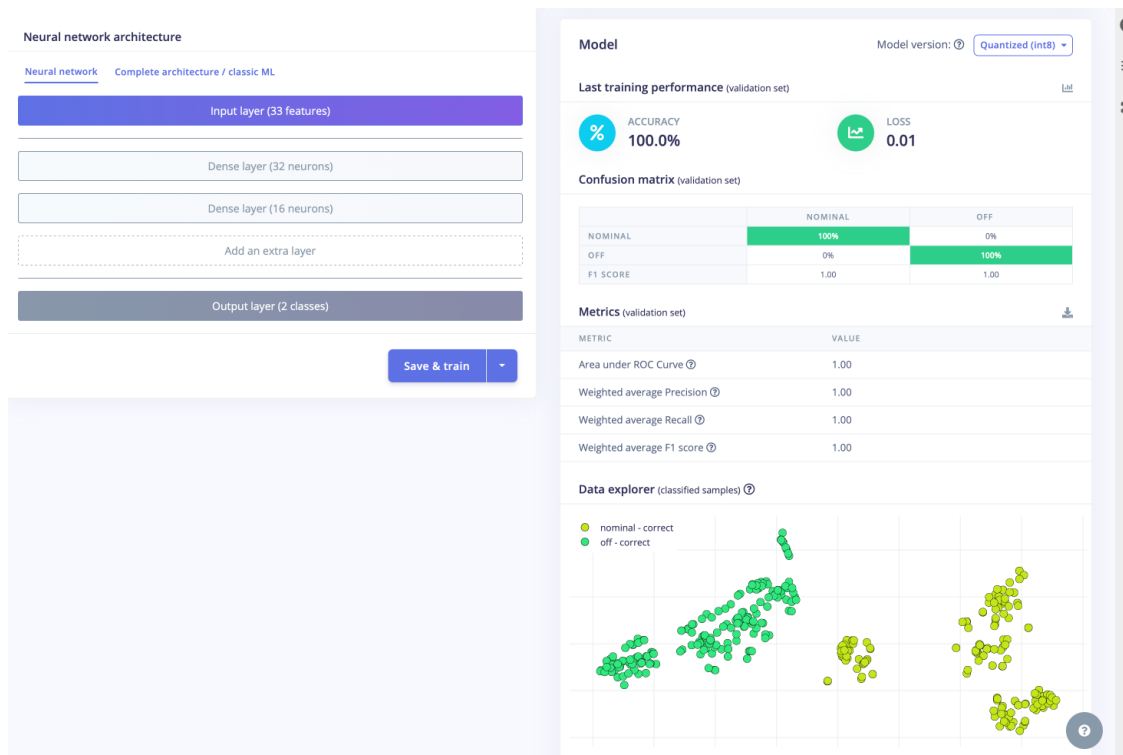
The classifier receives spectral features produced by the Spectral Analysis processing block rather than raw accelerometer samples. There are 33 input features in total, derived from the three axes. Five of the features used are `accX RMS`, `accX Peak 1 Height`, `accX Peak 1 Freq`, `accY RMS`, and

accY Peak 3 Height. These five are also the highest-ranked features in the feature importance output.



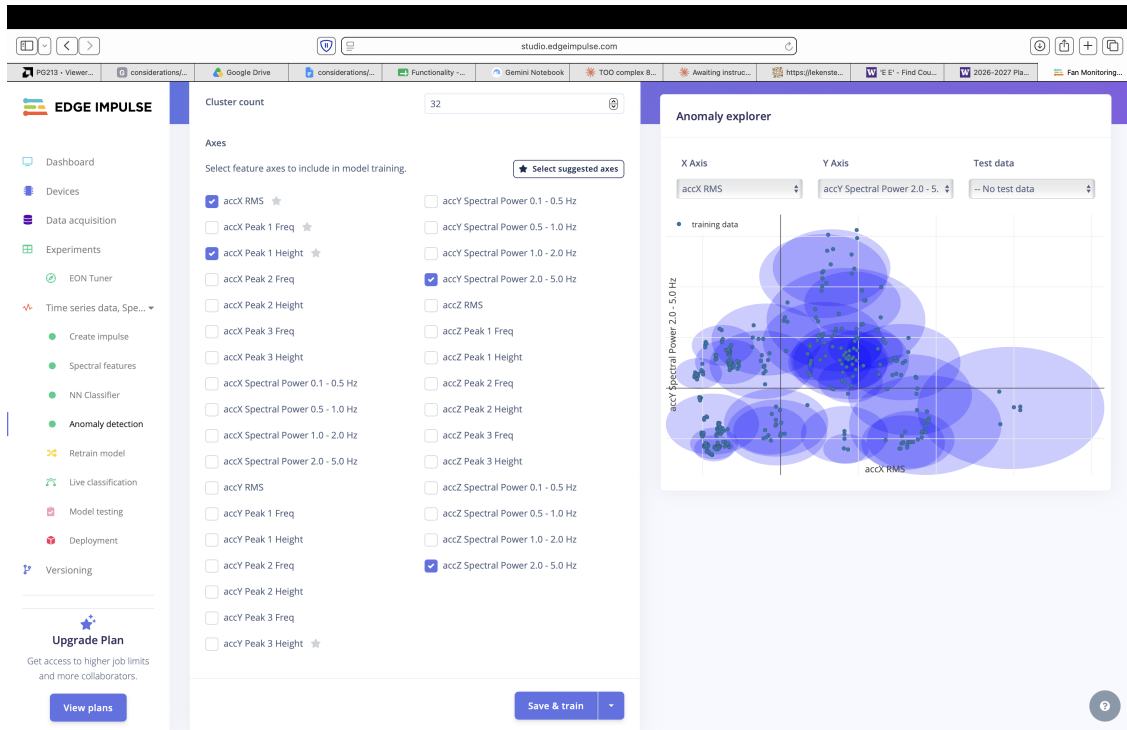
NN classifier outputs

The classifier outputs two classes, nominal and off. The int8 quantized model reached 100% validation accuracy with a validation loss of 0.01 and an F1 score of 1.00 for both classes.



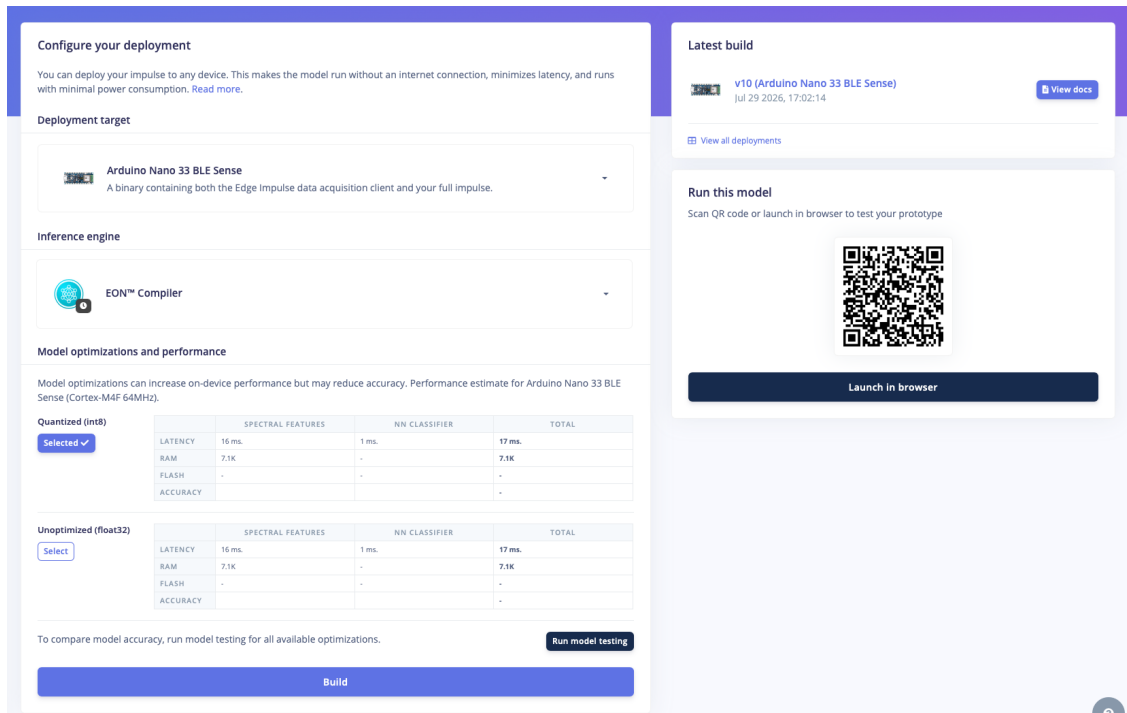
Anomaly detector features

The K-means anomaly detector also uses spectral features from the same processing block, but only a four-feature subset rather than all 33. The selected features are accX RMS, accX Peak 1 Height, accY Spectral Power 2.0 5.0 Hz, and accZ Spectral Power 2.0 5.0 Hz. The cluster count is set to 32.



Deployment evidence

The quantized int8 model was built as Arduino Nano 33 BLE Sense firmware with the EON Compiler enabled and flashed to the board. The estimated on-device performance for the int8 model is 17 ms total latency, of which 16 ms is spectral feature extraction and 1 ms is the classifier, using 7.1 KB of RAM.



```

Inferencing settings:
  Interval: 10.00 ms.
  Frame size: 600
  Sample length: 2000 ms.
  No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 611 us, anomaly 552 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.019531
  off: 0.980469
Anomaly prediction: 8.626719
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 45 ms, inference 581 us, anomaly 553 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500

```

Interpretation

With the board resting on a desk the classifier returned 0.4375 for nominal and 0.5625 for off, while the anomaly score was approximately 16.0. The classifier output should not be trusted here. A softmax output must sum to one, so the network always emits a class label even for input completely unlike anything it was trained on, and a near even split between the two classes carries essentially no information. The anomaly score is what signals that the input is unreliable: values in this range indicate the window fell far outside every one of the 32 training clusters, which is expected because the model was trained on a sensor mounted to a fan housing rather than a stationary board on a desk. The classifier output is only trustworthy when the anomaly score is low, meaning the current input resembles the training distribution.

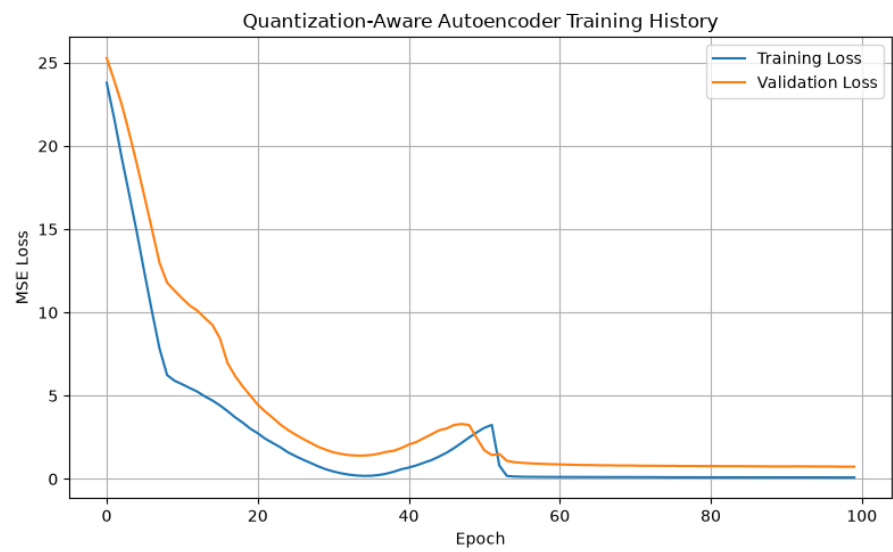
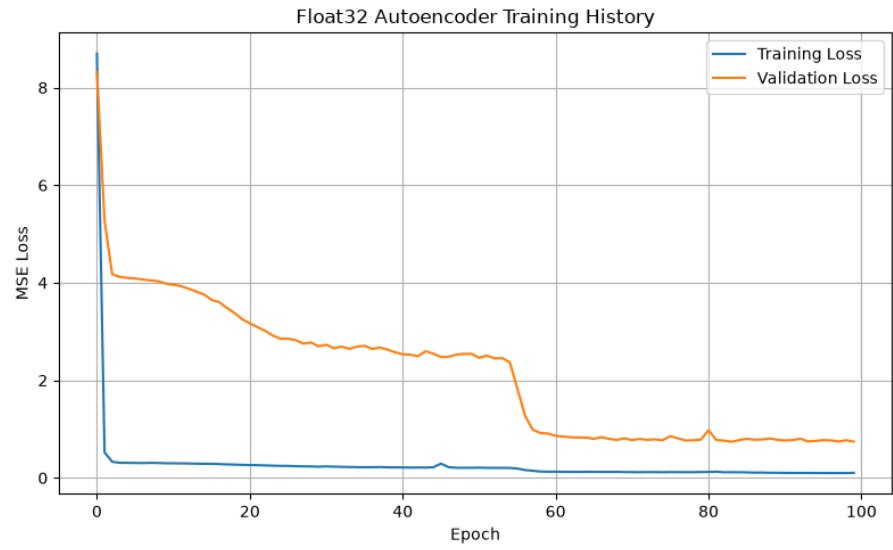
2 Autoencoder Model Results

Data preparation

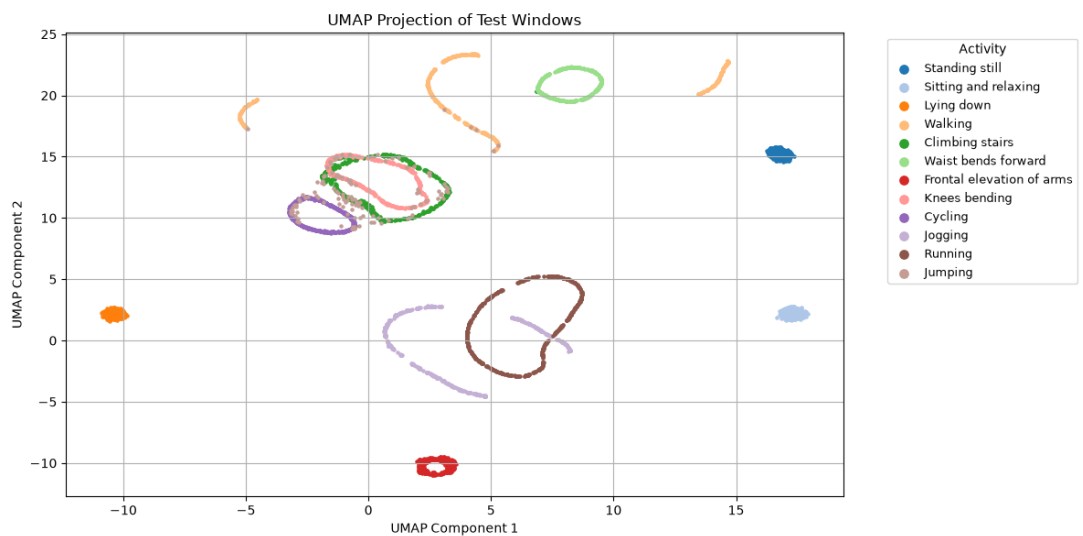
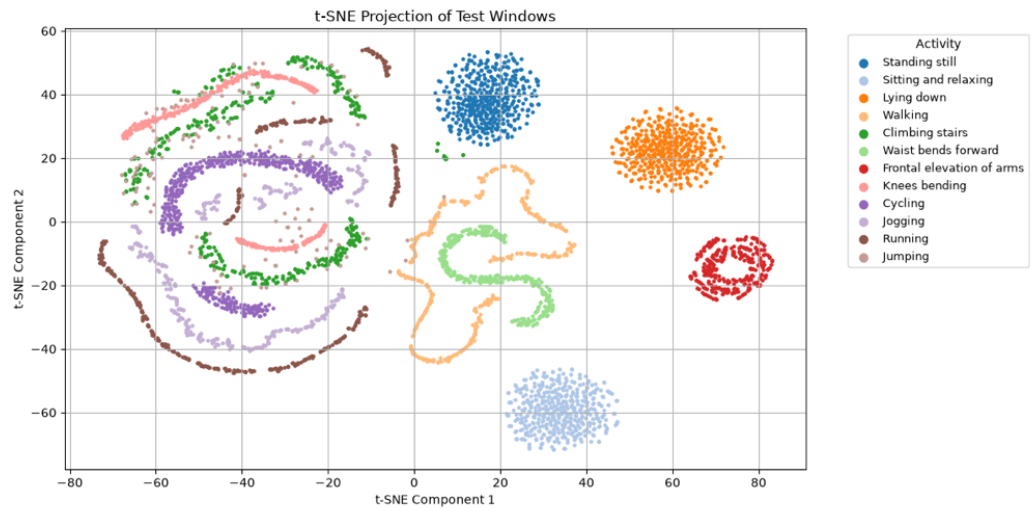
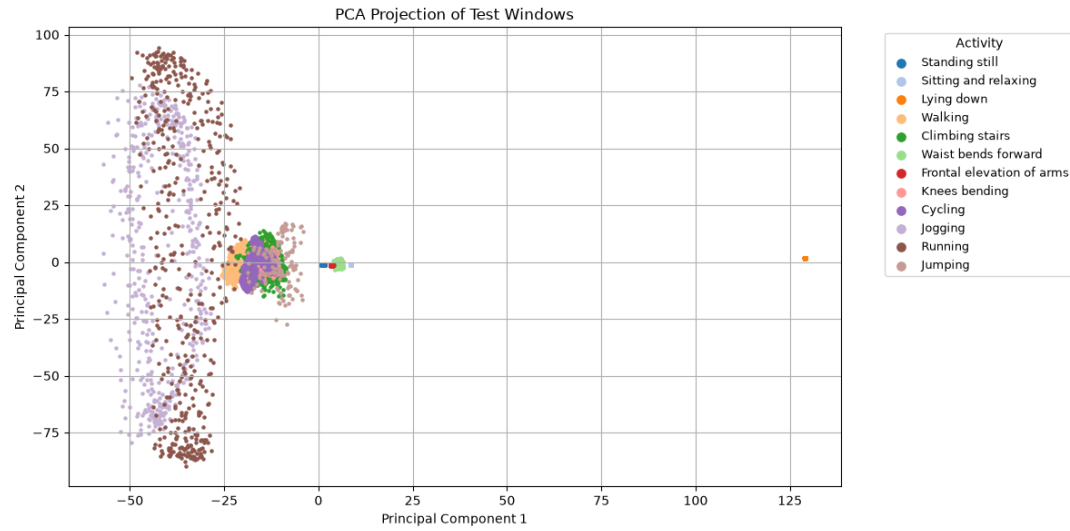
The left ankle accelerometer channels (columns 6, 7, and 8 of the raw file) and the activity label column (column 24) were used. Unlabeled rows were removed, and each activity segment was cut into 100-sample windows with a stride of 1, then flattened to 300-dimensional vectors. This produced 21,350 training windows and 8,479 test windows. Standing still, sitting and relaxing, lying down, waist bends forward, frontal elevation of arms, and knees bending were defined as the normal group, giving 10,478 normal and 10,872 anomalous training windows.

Training and validation loss

The autoencoder has a 300 32 16 32 300 architecture with 20,604 parameters and a linear output layer, trained only on the 10,478 normal windows. The float32 model reached a final validation loss of 0.743, and after quantization-aware training the model reached 0.739, so quantization did not degrade reconstruction quality.

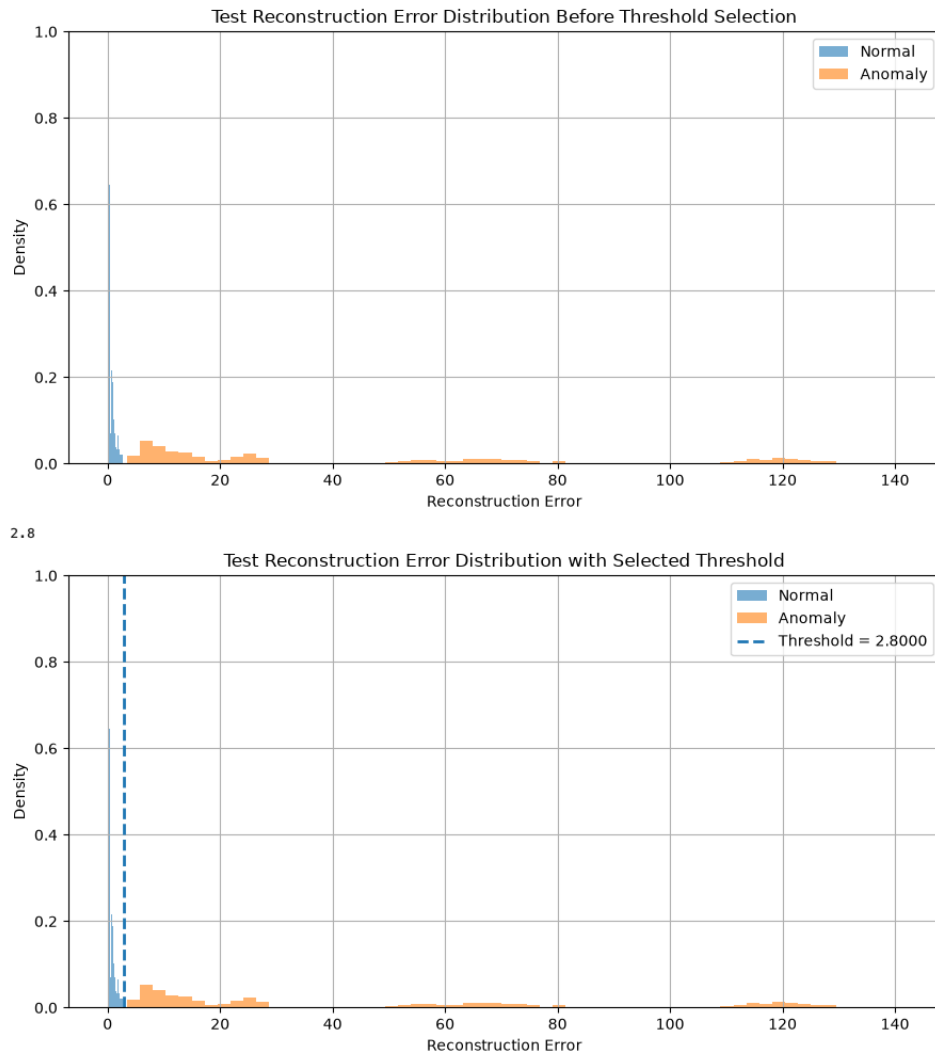


Dimensionality reduction



Both t-SNE and UMAP separate the static activities into compact isolated clusters while the periodic activities form loops and ribbons, which reflects the cyclic nature of walking, jogging, and running windows. PCA collapses most activities near the origin because two linear components are dominated by the highest-variance activities, which illustrates why the nonlinear methods are more useful for inspecting this data.

Reconstruction error and selected threshold

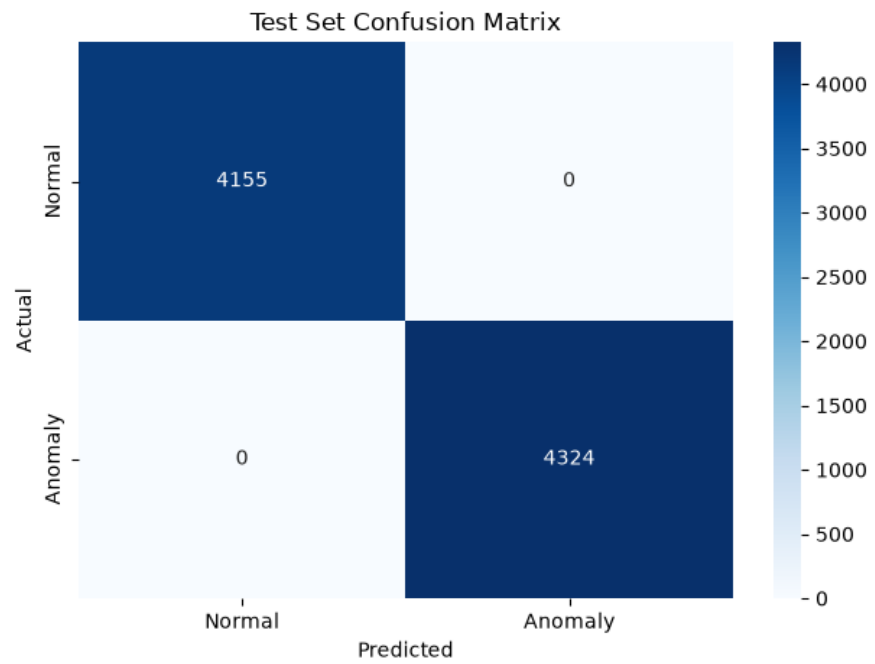
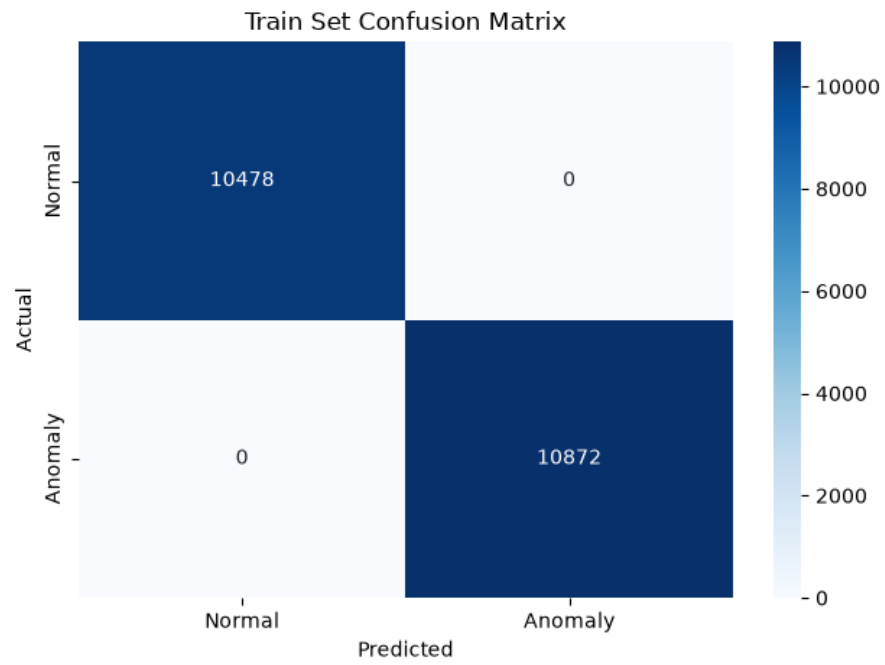


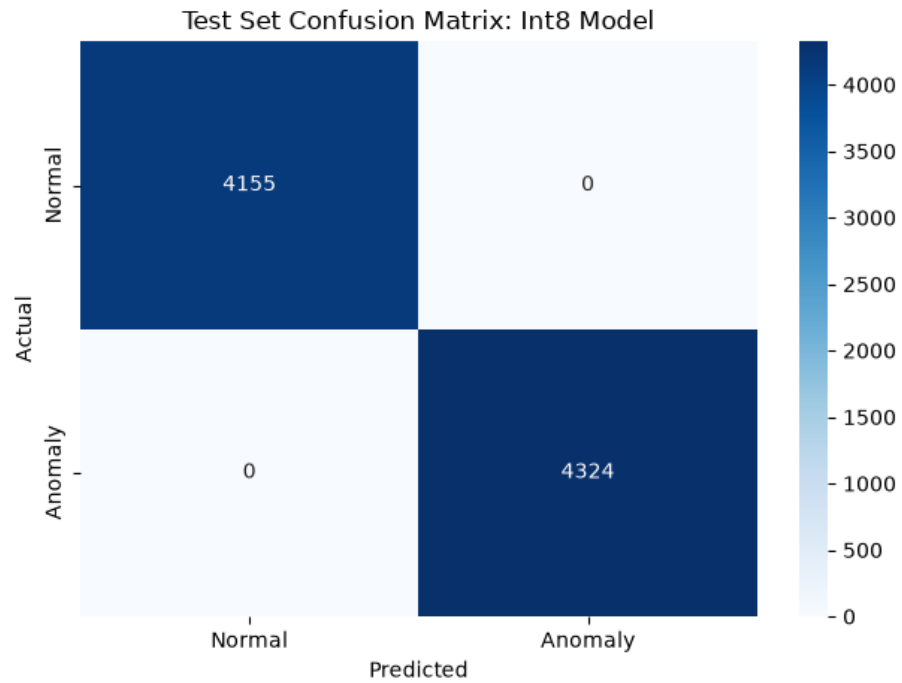
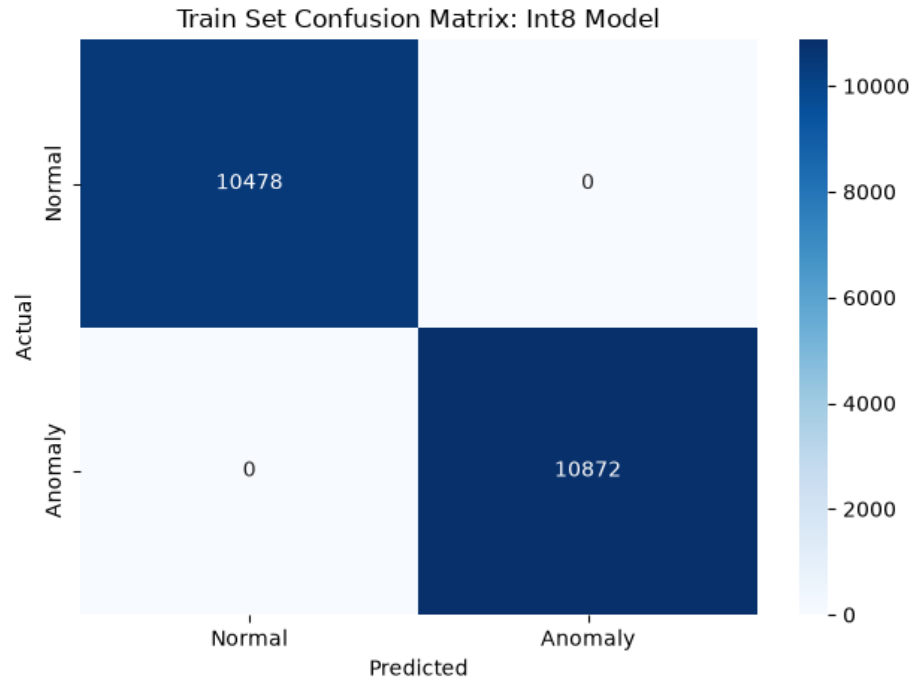
The threshold was set to 2.8. The two error distributions are disjoint with a wide margin: the largest normal training error is 1.690 while the smallest anomalous training error is 2.969, and on the test set the corresponding values are 2.700 and 3.465. A threshold of 2.8 sits inside that gap and was chosen from the training data alone. Normal errors average 0.158 and anomalous errors average 46.1, roughly a factor of 300 apart, so the decision is insensitive to the exact value within the gap.

Confusion matrix and classification report

Both the quantization-aware float32 model and the converted int8 model classify every window correctly on both splits, with precision, recall, and F1 all equal to 1.00 and no off-diagonal entries

in either confusion matrix.





Model size

The float32 TFLite model is 82.87 KB and the full int8 model is 24.46 KB, a reduction of roughly 3.4 times with no loss in accuracy. Estimated tensor memory is 21.90 KB, and the exported model is 25,048 bytes.

Arduino serial monitor output

```
.7:51:29.641 -> Reconstruction error: 0.344744  
.7:51:29.641 -> Result: Normal activity  
.7:51:29.675 -> Reconstruction error: 0.344744  
.7:51:29.675 -> Result: Normal activity  
.7:51:29.707 -> Reconstruction error: 0.345034  
.7:51:29.708 -> Result: Normal activity  
.7:51:29.740 -> Reconstruction error: 0.345476  
.7:51:29.740 -> Result: Normal activity  
.7:51:29.740 -> Reconstruction error: 0.345527  
.7:51:29.740 -> Result: Normal activity  
.7:51:29.772 -> Reconstruction error: 0.345646  
.7:51:29.772 -> Result: Normal activity  
  
17:47:47.074 -> Reconstruction error: 413.674713  
17:47:47.074 -> Result: Anomaly detected  
17:47:47.108 -> Reconstruction error: 412.931366  
17:47:47.108 -> Result: Anomaly detected  
17:47:47.140 -> Reconstruction error: 417.149017  
17:47:47.140 -> Result: Anomaly detected  
17:47:47.174 -> Reconstruction error: 419.614594  
17:47:47.174 -> Result: Anomaly detected
```

With the board held stationary in an orientation matching the training data, the reconstruction error was 0.345 and the result was reported as normal activity, consistent with the normal error range measured in Python. Shaking the board raised the error to approximately 415 and the result changed to anomaly detected. Intermediate orientations gave 3.84 with the board on edge and 10.32 with the board flat on the desk, both above the threshold while stationary, which shows that sensor orientation alone shifts the reconstruction error by more than an order of magnitude.

3 Discussion Questions

What threshold did you choose for anomaly detection, and why?

A threshold of 2.8 was chosen. The normal and anomalous reconstruction error distributions do not overlap, so the threshold was placed inside the empty gap between the largest normal training error of 1.690 and the smallest anomalous training error of 2.969. A percentile-based cut such as the 95th percentile of normal errors would have accepted a fixed five percent false positive rate that the data does not require, and in fact produced 560 false positives on the test set before being replaced.

How does reconstruction error help distinguish normal and anomalous activity?

The autoencoder is trained only on normal windows, so its 16-unit bottleneck learns a compressed representation that captures the structure of low-motion activity and nothing else. Normal windows pass through and are reconstructed closely, giving small errors, while windows from activities the model never saw cannot be represented by that bottleneck and are reconstructed poorly. The magnitude of the error therefore acts as a distance from the learned normal distribution, which requires no anomalous labels during training.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The model bytes themselves, the input and output quantization parameters, the reconstruction error threshold, and the input format must all be preserved. The input scale of 0.0814 with zero point 18 and the output scale of 0.0820 with zero point 6 are read from the model at runtime and are required to convert between float and int8 correctly. The threshold of 2.8 must be copied into the sketch, and the input must be a 300-element vector formed from 100 samples of three axes in the same interleaved order used during training.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The threshold is only meaningful in the units and coordinate frame the model was trained in. The mHealth data is in metres per second squared while the board's IMU library returns values in g, so the three readings must be multiplied by 9.81 before entering the window buffer, or every sample arrives roughly ten times too small and the error lands in a range the threshold was never calibrated for. Sensor placement matters just as much. The measurements above show the stationary error changing from 0.345 to 3.84 to 10.32 purely as a function of how the board was held, which means the same physical state can be reported as either normal or anomalous depending on orientation. The int8 input range also makes this concrete: with scale 0.0814 and zero point 18 the representable range is roughly -11.9 to $+8.9$ m/s², so an axis pointing such that gravity reads positive is clipped before inference even begins, because the representative dataset only ever contained ankle-mounted normal activity.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The threshold is a single fixed constant chosen offline, so the detector cannot adapt to a new sensor placement, a new user, or drift over time without being recalibrated and reflashed. The model only recognises normal activity as it appeared in the training set, so any unmodelled but harmless condition is reported as an anomaly, and the output is a binary decision with no indication of what kind of anomaly occurred. The int8 input range is fixed by the representative dataset and clips values outside it, which silently distorts inputs the model was not calibrated for. Memory is also a constraint: the model occupies about 24 KB with a 64 KB tensor arena, which limits how large a model or how long a window the board can support. Finally, the sketch keeps samples in a ring buffer but always reads it starting from index zero, so after the first window each input contains a discontinuity at the wrap point, which inflates the reported error relative to the notebook.